

Boldmaskine

For at underholde deltagerne ved IOI har Madina opfundet en boldmaskine. Boldmaskinen er indrettet på følgende måde:

- Maskinen består af N **knuder**, indekseret fra 0 til $N - 1$. Knuden $N - 1$ kaldes **roden**
- For hver knude u , hvor $0 \leq u < N - 1$, er **forældren** til knude u knuden $P[u]$, hvor $P[u] > u$, og knuden u er et **barn** af $P[u]$. Roden har ikke nogle forældre. En knude uden nogle børn kaldes et **blad**.

Du kender **ikke** N , eller forældren $P[u]$ for nogen af knuderne u . Til gengæld fortæller Madina dig M , antallet af blade, og at indekserne for bladene er $0, 1, \dots, M - 1$. Din opgave er at bestemme den indre struktur af maskinen ved at bruge maksinen.

Hver knude i maskinen kan højst holde én **bold**, og hver bold har en værdi som er et ikke-negativt heltal. Vi siger at en knude har værdien x hvis den holder en bold med værdi x . En knude siges at være **optaget** hvis den holder en bold; ellers er den **ledig**. Til at starte med er alle knuder ledige.

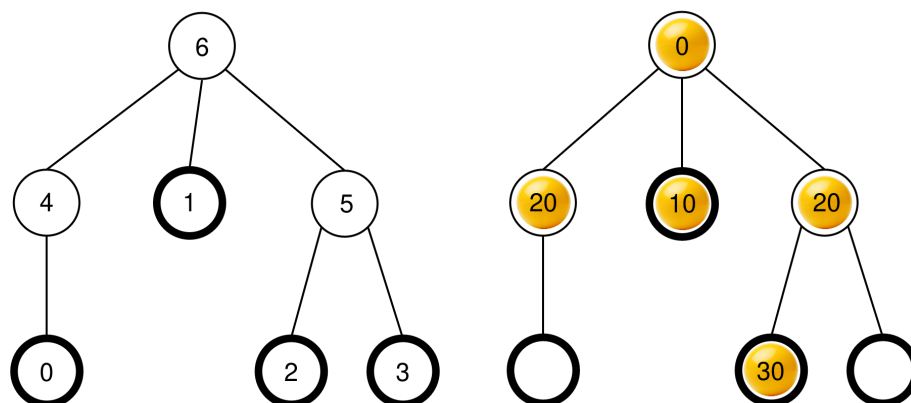
Du kan udføre to slags operationer:

- **insert** (U, X): Skriv værdien X på en ny bold og forsøg at indsætte den i bladet U .
 - Hvis bladet U er optaget returnerer operationen `false` uden at indsætte nogle bolde
 - Hvis bladet U er ledigt, placeres bolden i knude U . Derefter vil bolden bevæge sig til forældren af den nuværende knude, hvis forældren er ledig. Og så længe den nye forælder er ledig bevæger bolden sig op. Bevægelsen stopper når bolden når roden eller hvis forældren er optaget. Operationen returnerer `true`.
- **collect()**: Hvis roden er ledig (hvilket betyder at maskinen er tom), så vil `collect` returnere en tom liste. Ellers vil den rekursive funktion `traverse`, beskrevet nedenunder, samle værdierne af alle boldene der er i maskinen i en liste S . Til at starte med er listen S tom og `traverse(N - 1)` bliver kaldt.

```
traverse(u):  
    tilføj værdien af bolden ved knude u til enden af S  
    lad c[u] være en liste af de optagede børn af u  
    sorter c[u] i svagt stigende rækkefølge med hensyn til værdierne  
        af boldene i de tilsvarende knuder (hvis flere knuder har samme værdi  
        kan de fremkomme i en vilkårlig rækkefølge)  
    for hver v i c[u]:  
        traverse(v)
```

Efter at funktionen er færdig fjernes alle boldene fra maskinen, og `collect` returnerer S .

For eksempel. Betragt en maskine med $N = 7$ knuder og forældre liste $P = [4, 6, 5, 5, 6, 6]$. Billedet til venstre viser den tomme maskine med indekser, hvorimod billedet til højre viser en mulig opstilling af boldene efter en sekvens af `insert` operationer (Denne sekvens er givet i eksempel afsnittet).



Når `collect()` bliver kaldt er roden (knode 6) optaget, så S vil blive sat til $S = []$ og `traverse(6)` bliver kaldt.

traverse(6): knode 6 har værdien 0; denne værdi tilføjes til enden af S hvilket giver $S = [0]$. Børnene til knode 6 er 4, 1 og 5, disse knuder er alle optaget, og deres værdi er henholdsvis 20, 10 og 20. Sorterer vi dem i svagt stigende rækkefølge får vi $c[6] = [1, 5, 4]$ (hver opmærksom på at $c[6] = [1, 4, 5]$ også ville være korrekt, siden at knode 4 og knode 5 har samme værdi). Funktionen vil derefter kalde `traverse` på hver af knuderne i $c[6]$ i den rækkefølge.

- **traverse(1):** knode 1 har værdi 10; tilføjer vi den til slutningen af S får vi $S = [0, 10]$. Knude 1 har ikke nogle optagede børn, så funktionskaldet afsluttes.
- **traverse(5):** knode 5 har værdien 20; tilføjer vi den til slutningen af S får vi $S = [0, 10, 20]$. Knude 5 har et optaget barn, knode 2, så $c[5] = [2]$ og funktionen kalder `traverse(2)`.
 - **traverse(2):** knode 2 har værdien 30; tilføjer vi den til slutningen af S får vi $S = [0, 10, 20, 30]$. Knude 2 har ikke nogle optagede knuder så funktionskaldet afsluttes.
 - Udførelsen af funktionen returnerer til `traverse(5)` som har behandlet alle børnene i $c[5]$, så funktionskaldet afsluttes.
- **traverse(4):** knode 4 har værdi 20; tilføjer vi den til slutningen af S får vi $S = [0, 10, 20, 30, 20]$. Knude 4 har ikke nogle optagede børn, så funktionskaldet afsluttes.

Alle funktionskald er blevet håndteret og der er ikke flere knuder at behandle. Til slut fjernes alle bolde fra maskinen og `collect` returnerer listen $S = [0, 10, 20, 30, 20]$.

Din opgaver er at bestemme antallet af knuder N og returnere en liste af forældre $R = [R[0], R[1], \dots, R[N-2]]$ der beskriver strukturen af maskinen, men måske med en eventuel anden afmærkning af de knuder, der hverken er blade eller roden. Formelt betragtes en liste R

som en korrekt forældreliste hvis det er muligt at lave en unik afmærkning $L[u]$ med $0 \leq L[u] < N$ til hver knude u af maskinen så følgende betingelser er overholdt:

- $L[u] = u$ for alle $0 \leq u < M$ og $u = N - 1$, og
- $L[P[u]] = R[L[u]]$ for alle $0 \leq u < N - 1$.

Det er **ikke** nødvendigt at $R[u] > u$. Maskinen **skal være tom** når du returnere din løsning.

Lad K være antallet af `collect` operationer der bliver udført, og B være den maksimale værdi af boldene over alle `insert` kald. Lad $C = K + B$. Så må C **ikke overstige** 1000. Dine point i nogle delopgaver vil afhænge af værdien C .

Implementerings detaljer

Du skal implementere følgende funktion.

```
std::vector<int> find_structure(int M)
```

- M : antallet af blade i maskinen.
- Funktionen kaldes præcis en gang for hver test case.
- Funktionen skal returnere en liste $R = [R[0], R[1], \dots, R[N - 2]]$ af længde $N - 1$, en korrekt forældreliste.

For at bruge maskinen kan din funktion kalde følgende to funktioner.

Den første funktion er:

```
bool insert(int U, int X)
```

- U : indekset på bladet hvor bolden skal indsættes. Der skal gælde at $0 \leq U < M$.
- X : heltalsværdien af bolden. Der skal gælde at $0 \leq X \leq 1000$.
- Denne funktion returnerer `true` hvis indsættelsen af bolden var vellykket, eller `false` hvis bladet U allerede var optaget.
- Denne funktion kan højst blive kaldt 500 000 gange.

Den anden funktion er:

```
std::vector<int> collect()
```

- Indsaml værdierne på alle indsatte bolde, tøm maskinen og returner den indsamlede liste S .

Hvis C på noget tidspunkt overstiger 1000 igennem kørslen af programmet vil din løsning modtage resultatet `Output isn't correct: Too many resources used`.

Strukturen af maskinen er fastlagt inden `find_structure` bliver kaldt.

Begrænsninger

- $2 \leq N \leq 1000$
- $1 \leq M \leq 200$
- $M < N$

Delopgaver

Delopgave	Point	Yderligere begrænsninger
1	5	Roden har præcis M børn.
2	10	$M \leq 3$
3	25	$N \leq 200, M \leq 45$
4	60	Ingen yderligere begrænsninger.

I subtasks 3 og 4, afhænger dine point af værdien C som beskrevet nedenunder.

Delopgave 3 (25 points)

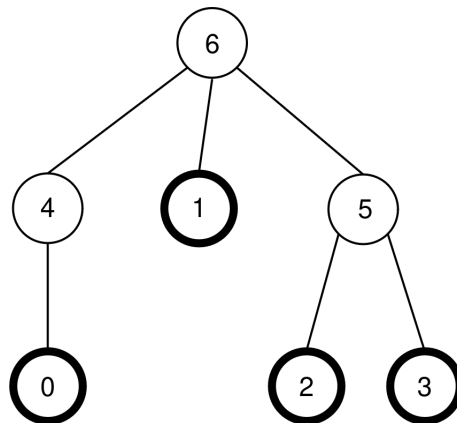
Begrænsninger	Point
$1000 < C$	0
$45 < C \leq 1000$	13
$C \leq 45$	25

Delopgave 4 (60 points)

Begrænsninger	Point
$1000 < C$	0
$200 < C \leq 1000$	7
$71 < C \leq 200$	$47 - \frac{C}{5}$
$44 < C \leq 71$	$104 - C$
$C \leq 44$	60

Eksempel

Betragt den samme maskine som i opgavebeskrivelsen, så $N = 7$, $M = 4$, og $P = [4, 6, 5, 5, 6, 6]$. Maskinen er igen vist i følgende figur:



Graderen laver følgende kald:

```
find_structure(4)
```

Funktionen kan lave følgende sekvens af kald:

1. **insert(0, 0)**: blad 0 er ledigt, så en bold med værdi 0 bliver placeret i blad 0. Knuden $P[0] = 4$ er ledig, så bolden bevæger sig til knude 4. Knuden $P[4] = 6$ er ledig, så bolden bevæger sig til knude 6. Siden knude 6 er roden bevæger den sig ikke videre. Funktionskaldet returnerer `true`.
2. **insert(3, 20)**: blad 3 er ledigt, så en bold med værdi 20 bliver placeret i bladet 3. Knuden $P[3] = 5$ er ledig, så bolden bevæger sig til knude 5. Siden $P[5] = 6$ er optaget bevæger den sig ikke videre. Funktionskaldet returnerer `true`.
3. **insert(1, 10)**: blad 1 er ledigt, så en bold med værdi 10 bliver placeret i blad 1. Siden $P[1] = 6$ er optaget vil bolden blive i knude 1. Funktionskaldet returnerer `true`.
4. **insert(2, 30)**: bladet 2 er ledigt, så en bold med værdi 30 bliver placeret i blad 2. Siden $P[2] = 5$ er optaget vil bolden blive i knude 2. Funktionskaldet returnerer `true`.
5. **insert(1, 25)**: blad 1 er optaget, så bolden vil ikke blive placeret i blad 1. Funktionskaldet returnerer `false`.
6. **insert(0, 20)**: blad 0 er ledigt, så en bold med værdi 20 bliver placeret i blad 0. Knuden $P[0] = 4$ er ledig, så bolden bevæger sig til knude 4. Siden $P[4] = 6$ er optaget bevæger den sig ikke videre. Funktionskaldet returnerer `true`.

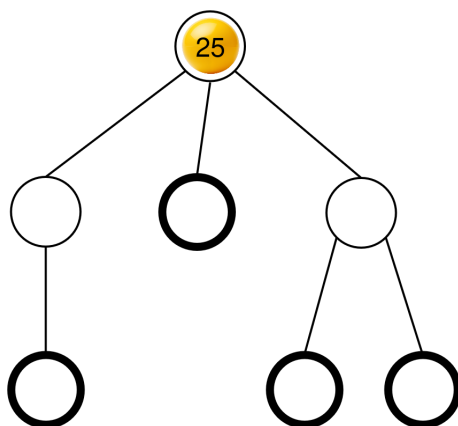
Den resulterende opstilling af boldene var allerede vist i opgavebeskrivelsen.

Derefter vil funktionen kalde:

```
collect()
```

Kørslen af funktionskaldet var allerede beskrevet i opgavebeskrivelsen, så funktionskaldet returnerer $S = [0, 10, 20, 30, 20]$. Derefter vil maskinen igen være tom.

Derefter kan funktionen kalde `insert(2, 25)`. Blad 2 er ledigt, så en bold med værdi 25 bliver placeret ved blad 0. Denne bold bevæger sig til knude 5, og derefter til knude 6 hvor den vil stoppe. Funktionskaldet returnerer `true`. Den resulterende opstilling af bolde er vist i den følgende figur.



Til sidst kan funktionen kalde `collect()` hvilket returnerer $S = [25]$ og tømmer maskinen.

Der er to korrekte forældrelistes som `find_structure` kan returnere: $R = P = [4, 6, 5, 5, 6, 6]$ (hvilket svarer til afmærkningen $L = [0, 1, 2, 3, 4, 5, 6]$), og $R = [5, 6, 4, 4, 6, 6]$ (hvilket svarer til afmærkningen $L = [0, 1, 2, 3, 5, 4, 6]$). I dette eksempel har vi, $K = 2$ og $B = 30$, så $C = 32$.

Eksempel Grader

Input format:

```
N M
P[0] P[1] P[2] ... P[N-2]
```

Output format:

```
K B C
Q
R[0] R[1] R[2] ... R[Q-1]
```

Her er Q længden af listen R returneret af `find_structure`.